

Makefile

Exemple d'un **makefile** multi-répertoire

Ce tutoriel présente un *makefile* qui peut être utilisé pour la création de projets en C. Il est présenté section par section.

1 Configuration

Dans la partie configuration, il y a trois variables à renseigner. La première, nommée EXEC, correspond au nom de l'exécutable (pas d'extension sous *Linux*). S'il y a plusieurs executables, il faut spécifier les différents noms, séparés par un espace. Par exemple :

```
EXEC = test1 test2
```

La deuxième, nommée OBJECTS, correspond aux objets créés (**ils doivent être indiqués avec l'extension .o** et doivent être séparés par des espaces). Chaque objet correspond à un fichier source (.c) et éventuellement à un fichier d'en-tête (.h). Par exemple :

```
OBJECTS = example1.o example2.o example3.o
```



Dans la variable OBJECTS, vous ne devez pas spécifier l'objet associé à l'exécutable.

La troisième, nommée PROJECT_NAME, est utilisée pour identifier le nom du projet en cours et pour l'archivage du projet.

Les quatre variables suivantes SRC_DIR, OBJECTS_DIR, INCLUDE_DIR et BIN_DIR indiquent respectivement les noms des répertoires pour les sources (.c), les objets (.o), les fichiers d'en-tête (.h) et les executables.

La section ARGUMENTS AND COMPILER correspond au compilateur utilisé et aux options passées en paramètres. La variable CC est le nom du compilateur. Par défaut, il s'agit de gcc. La variable CCFLAGS_STD est la liste des options spécifiées au compilateur :

- -Wall : pour afficher tous les avertissements (*warning*).
- -O3 : pour optimiser la compilation (la compilation est plus longue mais la vitesse d'exécution est très rapide).
- -Werror : pour considérer tous les avertissements comme des erreurs (dès qu'un avertissement est indiqué, la compilation échoue).

La variable CCFLAGS_DEBUG correspond aux options spécifiées au compilateur dans le cas du mode "DEBUG". La variable CCLIBS peut être utilisée pour lier des bibliothèques. Par exemple, avec la bibliothèque *ncurses*, on peut écrire CCLIBS = -lncurses.

2 Gestion des dépendances et compilation

Avant de compiler, il est nécessaire de spécifier les dépendances. Sans cela, les sources ne seront pas compilées ou ne seront pas recompilées en cas de besoin. **Il ne faut pas le faire manuellement.** Il faut utiliser la règle `depend`. Elle crée automatiquement toutes les dépendances qui sont ajoutées à la fin du fichier dans la section `DEPENDENCIES`. Pour exécuter la règle, tapez la commande suivante :

```
make depend
```



La règle `depend` doit être exécutée à chaque fois qu'un objet est ajoutée dans le `makefile`, ainsi qu'à chaque fois que de nouvelles inclusions sont ajoutées dans le projet (uniquement les inclusions de fichiers d'en-tête utilisateur).

Une fois les dépendances ajoutées, vous pouvez compiler les programmes en tapant la commande suivante :

```
make
```

3 Autres règles

Pour nettoyer le projet et supprimer tous les fichiers temporaires et les objets, vous pouvez utiliser la règle `clean`. Si le projet génère des fichiers spécifiques que l'on souhaite supprimer, il est possible de les ajouter à la suite. Pour exécuter la règle, tapez la commande suivante :

```
make clean
```



La règle `clean` a été proposée pour nettoyer tout le projet. Seuls les objets (fichiers `.o`) et les exécutables sont supprimés, ainsi que les fichiers temporaires. Si les dépendances ont bien été faites (avec la règle `depend`), **il ne faut pas utiliser cette règle systématiquement** avant chaque compilation. Le cas échéant, le `makefile` ne servirait à rien.

Pour sauvegarder le projet, vous pouvez utiliser la règle `archive`. Une archive (au format `.tar.gz`) est créée contenant le contenu de tout le répertoire courant (à noter que cette règle appelle la règle `clean` au préalable). L'archive est placée dans le répertoire parent. Le nom de l'archive dépend du nom du projet (variable `PROJECT_NAME`) et il est suivi de la date du jour. Pour exécuter la règle, tapez la commande suivante :

```
make archive
```

4 Débogage manuel

Lors du développement (en C), il est parfois nécessaire de mettre en place des parties qui devront disparaître dans la version finale du programme. Une solution consiste à définir une constante quelconque (ici, le choix a été fait d'utiliser `_DEBUG_`). Dans les sources, les parties à réaliser uniquement en mode débogage sont spécifiées comme suit :

```
#ifndef _DEBUG_  
    /* Code à réaliser en mode debug */  
#endif
```

Si la constante n'a pas été définie, les instructions seront ignorées. Pour compiler en mode débogage à l'aide du `makefile`, il suffit de taper la commande suivante :

```
make debug
```

Les options de compilation définies dans la variable `CCFLAGS_DEBUG` du `makefile` seront ajoutées lors de la compilation. Avec `gcc`, il suffit d'ajouter `-D _DEBUG_`.



Si le programme a déjà été compilé et que vous souhaitez réaliser une compilation en mode débogage, il faudra d'abord faire appel à la règle `clean`. Procédez de même si vous avez compilé en mode débogage et que vous souhaitez compiler sans ce mode.